An odyssey on ST's SensorTile.box PRO

Field notes from building an open-source movement logger for foiling — STM32U585, BlueNRG-LP, ThreadX/FileX, BLE, GPS, and a cross-platform Rust desktop GUI. May 2026.

> **Source & downloads.** Firmware + visualizer + GUI on GitHub: github.com/zdavatz/fp-sns-stbox1 — based on ST's FP-SNS-STBOX1 function pack (BSD-3-Clause), our additions where original under MIT.

What we set out to do

Build a self-contained *movement logger* for pump foiling: a small box on the mast that records inertial data, magnetometer, barometer, GPS and audio to an SD card; sync the files to a desktop GUI over Bluetooth; visualize the ride. Open source end-to-end, no cloud.

The hardware target was STMicroelectronics' **STEVAL-MKBOXPRO** (the "SensorTile.box PRO"): STM32U585 host, BlueNRG-LP BLE controller, LSM6DSV16X IMU, LIS2MDL magnetometer, LPS22DF barometer, STTS22H thermometer, MEMS mic, microSD slot, USB-C, LiPo charger — packaged in a 50 × 35 mm enclosure with a piezo buzzer, LEDs, and two buttons. ST ships an MCU package with HAL + RTOS + BLE examples. Perfect starting point.

"Perfect" is the dangerous word. What follows is the highlight reel of where reality diverged from the datasheets — and how an open codebase plus one very patient field tester got us through.

Mod #1: the 3.3 V rail

The box natively drives sensors at 1.8 V to save power. We wanted to add an external u-blox MAX-M10S GPS module, which is firmly 3.3 V. So we lifted the rail. The mod worked perfectly for a year — until BLE was switched on, at which point everything fell over. The chip wasn't drawing more current; the hardware mod was a red herring (more on that below).

The SD logging stack

Logging at 100 Hz sensors + 10 Hz GPS + 16 kHz audio + 1 Hz battery to FAT32 over SDMMC is not glamorous, but it's where most of the firmware sweat lives. Azure RTOS ThreadX + FileX (also from ST, also free) does the heavy lifting. A few unglamorous decisions that *must* stay set after every CubeMX regenerate:

- **SDMMC kernel clock at NSPEED (~12.5 MHz), not HSPEED (~25 MHz).** Halves the bandwidth ceiling to ~6 MB/s, which is still 1000× the logger's budget, and buys back signal-integrity margin on the 3.3 V-modded rail.
- **UART4 IRQ pushes raw bytes, parses in thread.** NMEA assembly used to run in `HAL_UART_RxCpltCallback` at NVIC priority 6 — above SDMMC. At 10 Hz GPS, SDMMC's budget blew out after ~90 s, the message queue filled, the sample rate collapsed from 100 Hz to ~77 Hz. Moving the parser to thread context fixed it cold.
- **I²C4 timing 0xA040184A** (~145 kHz) for the STC3115 fuel gauge. CubeMX's default 0x00F07BFF (~421 kHz) overshoots STC3115 Fast-Mode and the chip NAKs every other transfer.

- `fx_media_flush` **every 100 sensor samples** (≈ once per second). Without periodic flushes, FAT directory entries only update on file close — so power-off mid-session leaves zero-byte files on disk. With them, the worst case is one second of data lost.

The "BLE on, SD writes hang" odyssey (Issue #12)

This one took weeks. With `STBOX1_ENABLE_BLE_SYNC=1`, the SD writer wedged deterministically on `fx_file_create("Sens000.csv")`. Disable the flag → SD logging perfect. Re-enable → hung.

The obvious theory was: *BLE preempts SDMMC at NVIC priority X, corrupts state, hangs FAT.* That theory was wrong. We pinned EXTI11 (BLE HCI IRQ) to priority 14, same as SDMMC. Still hung. We bounded the BLE ISR to 16 events per IRQ. Still hung. We added a thread that parked *before* calling `bluetooth_init`. Still hung.

A field tester — Peter, owner of the only 3.3 V-modded board in active use — walked us through a six-build bisect over three days:

Build	BLE	VddIO2 enable	SD logging
v6 / v8	off	on	hung
v14	on	off	hung
v16	off	off	**works**

Two independent failure modes. Both had to be off for the field box to log.

The first culprit was a single line in `HAL_MspInit`: `HAL_PWREx_EnableVddIO2()`. Required by BLE on Rev_C (PG[15:2] is gated behind a separate I/O supply), it always ran — even when the BLE compile flag was off. On a stock 1.8 V board it is harmless; on the 3.3 V-modded board, with BLE not actually consuming the rail, the extra rail load was enough to perturb SDMMC. The BLE-flag gate had been incomplete.

The second culprit is still being narrowed (a knob, `STBOX1_BLE_BISECT_STAGE`, walks through chip-alive probe / `bluetooth_init` / EXTI11 enable, one stage at a time). What this odyssey already taught us:

- A compile flag named `ENABLE_BLE` doesn't mean "nothing related to BLE runs." Audit every site that touches BLE-relevant hardware (power rails, GPIO banks, clocks) and gate them too.
- One marginal hardware mod can mask *and* unmask completely unrelated bugs. Test on stock and modded units in parallel.
- If you have a field tester willing to flash six builds and read back error logs, you have a luxury most embedded shops don't. Cherish them.

The one-line fix that took a week: `EXTI11_IRQHandler`

Symptom: every variant of the BLE bring-up code wedged the moment BlueNRG-LP raised its HCI IRQ. USB heartbeat stopped, no thread ran, no further trace, no crash dump.

Cause: `stm32u5xx_it.c` had a strong handler for `EXTI13_IRQHandler` (user button) but *no* strong override of `EXTI11_IRQHandler`. GCC's startup file weak-binds unhandled interrupts to `Default_Handler`, which is literally:

```
Default_Handler:
    b Default_Handler
```

An infinite loop in assembly. The moment EXTI11 fired, the CPU vectored in and never came out. The fix was three lines — declare an `EXTI11_IRQHandler` that calls `HAL_EXTI_IRQHandler(&H_EXTI_11)`. A reference implementation in `BLEDualProgram` in the same repo had it; we'd copy-pasted everything *except* the line that mattered.

Advertising name truncation: `PumpTsueri` → `PumpTsu`

We chose `PumpTsueri` as the advertised BLE name (a nickname for the pump-foil device). On the phone, the box showed up as `PumpTsu`. The middleware (`Middlewares/ST/STM32_BLE_Manager/Src/ble_manager.c`) had hardcoded the advertising packet's name field at 7 bytes. We widened the struct (8 → 16), rewrote `set_connectable_ble` as a dynamic-length packet builder, rebuilt.

Still `PumpTsu`. The Makefile in the SDDataLogFileX application didn't emit `-MMD -MP` dependency files, so the header change rebuilt only the `.c` files we'd also touched. `ble_function.c` linked against the *old* struct layout and wrote `board_id` at the old offset; `ble_manager.c` read from the *new* offset and got zero. `make clean` fixed it. Lesson: ship dependency tracking from day one, even when the project "doesn't need it yet."

USB CDC console, killed by macOS Sequoia

To iterate faster than "DFU-flash, eject SD, read log, repeat", we brought up a USB CDC ACM virtual COM port over STM32U585's OTG_FS using TinyUSB. Heartbeat at 1 Hz prints ThreadX tick, OTG_FS IRQ count, BLE bring-up phase, USB state. Linux loves it — `cdc_acm` auto-attaches, `cat /dev/ttyACM0` streams.

macOS Sequoia 15 does not. The kernel CDC driver `AppleUSBCDCCompositeDevice` partial-attaches in `IOMatchDefer = Yes` state and never registers `/dev/tty.usbmodem*`. `libusb` can't claim the bulk endpoint either (the "accessory authorization" framework silently blocks vendor-class devices with no prompt). Tried CDC with IAD, pure CDC, vendor class — all blocked. Apparently a regression in Apple's DriverKit-era USB stack that's been hitting TinyUSB users broadly.

Workaround: keep a Linux laptop on the bench for live debug. Production builds default the flag off; we no longer attempt to use the USB console on macOS.

Bonus mini-stories

- **Soft-DFU via TAMP backup register.** Writing `DFU\n` to the CDC port reboots the chip into the STM32 system bootloader with no BOOT0 + slider gymnastics. Implementation: stash magic `0xDEADBEEF` in `TAMP->BKP0R`, soft-reset, check magic before `HAL_Init`, jump to `0x0BF90000`. A direct branch from running firmware leaks too much state on STM32U5; the bootloader sees a clean post-reset chip and brings up DFU cleanly.
- **Mode-switch via the same TAMP trick.** Box boots into BLE mode and advertises. iPhone GUI sends "start log" → firmware writes magic + duration to BKP1R/BKP2R, resets, comes up in LOG

mode with BLE off, logs for N seconds, resets back. BLE and SDMMC never run concurrently — neat sidestep of the Issue #12 root cause while it's still being narrowed.

- **Visualizer in pure Rust.** `stbox-viz` reads the on-card CSVs, runs Madgwick 6-DoF fusion, 4th-order Butterworth + filtfilt, scipy-equivalent STFT via `rustfft`, and emits interactive Plotly HTML (Scattermap + 5 stacked panels) or animated GIFs with a software-rasterised 3D foil mesh. One binary, no Python runtime, builds on macOS / Linux / Windows from one workspace.
- **Cross-platform GUI:** `MovementLogger`. egui drag-and-drop wrapper around the visualizer, plus a BLE FileSync panel (`btleplug` + tokio worker) that lists / reads / deletes files on the box without ejecting the SD card. Signed and notarized for macOS; the workspace patches `winit` to no-op `set_blur` because eframe → winit 0.30 calls a private CoreGraphics API (`_CGSSetWindowBackgroundBlurRadius`) that Apple's binary scanner rejects. Notarize the `.app` first, not just the DMG — Gatekeeper looks for the ticket on the app itself when users drag it out.

What we'd tell our past selves

- **Audit your weak symbols.** A missing strong handler is a silent infinite loop. Grep every IRQ name in your NVIC config against the actual handlers in `*_it.c`.
- **One compile flag is rarely enough.** "Disable feature X" means: gate the code path, the hardware init, the power rails, the GPIOs, the clocks. Build a minimal repro with the flag off and inspect what still differs from the no-feature baseline.
- **Burn build numbers on every invocation.** Our Makefile bumps `.build_counter` even on clean / failed builds, and bakes the number into the binary filename, the SD-card `FW_INFO_v<N>.TXT` and the error log boot marker. After a six-build bisect this is the difference between "Peter's third firmware that morning" and "build 254".
- **If you can move parsing out of an IRQ, do.** Especially on RTOS systems. The IRQ does one job: shovel bytes into a ring buffer. The thread parses at thread priority.
- **Field testers are co-authors.** A bug that reproduces only on one modded board, only with one peripheral combo, requires someone willing to flash, observe, photograph the LED morse, and ship back the error log. Write the firmware so their feedback channel is rich (boot markers, reset-reason decode, per-stage status bytes in a backup register surfaced to LEDs / buzzer / SD log) and they will carry the project further than any debugger.

Try it

The complete codebase — firmware, visualizer, GUI, documentation — is on GitHub:

github.com/zdavatz/fp-sns-stbox1

Pre-built MovementLogger binaries (signed for macOS and Windows) are attached to each release. Firmware `.bin` files for DFU flashing live alongside them. PRs welcome.